A 2D Graphics Game on an FPGA

Kishor Kunal, Soumyajit Halder

July 5, 2025

Figure 1: A sample image

Contents

Abstract

This project presents the design and implementation of a 2D graphics engine developed entirely in **Verilog** HDL, targeting the **Boolean Board** powered by a **AMD Spartan-7** FPGA.

The primary objective is to create a fully functional endless runner-style game while laying the groundwork for a scalable hardware game engine. The system architecture emphasizes modularity, performance, and resource efficiency, making it suitable for real-time operation on resource-constrained FPGA platforms. Through this work, we explore low-level graphics rendering, hardware-accelerated animation, and game logic entirely in HDL, offering insight into how real-time interactive systems can be realized on programmable logic devices.

System Overview

The architecture of the game engine is organized into modular subsystems, each handling a core aspect of functionality. This modularity enables scalability and independent testing, while also facilitating reuse in other game designs. The primary components include:

- **VGA Controller:** Generates display timing signals according to **VESA** standards, supporting a resolution of **640×480 at 60 Hz**.
- **Scrolling Background System:** Implements an infinite background, using limited memory.
- **Tile Management System:** Handles 32×32 pixel tiles with 4-bit color indexing, efficiently stored and retrieved from BRAM.
- **Rendering Pipeline:** Utilizes FIFO buffers and asymmetric dual-port memory to efficiently manage and composite tile and sprite data in real time.
- **Procedural Level Generator:** Dynamically generates terrain and platform layouts using pseudo-random sequences while maintaining game logic constraints.
- **Audio Output System:** Capable of producing sound effects and simple background music through waveform synthesis.
- **UART Input Handler:** Receives player control inputs from a PC keyboard via UART serial communication.
- **Sprite Animation System:** Controls character animation states such as idle, running, jumping, and crouching using pipelined frame rendering.
- **Physics Engine:** Simulates basic physics such as gravity, collision detection, and character-environment interactions.
- **JavaScript Tile Designer Tool:** A web-based utility to design tile maps and sprite frames, exporting memory initialization files (.COE) for use in the FPGA.

Each of these subsystems is described in detail in the following sections.

1 VGA Controller

The VGA controller is responsible for generating synchronization and timing signals to correctly display video on a monitor. Although the Boolean Board does not have a native VGA port, it includes an HDMI port. We use a VGA-to-HDMI converter IP to transform the VGA-style signal output to HDMI for compatibility with modern displays.

This project implements the **VGA 640×480 @ 60 Hz** timing standard based on the VESA Specification 1.13. Below is a summary of the key timing values extracted from the specification:

Horizontal Timing (in pixels)

Visible Area (Active Video)	640
Front Porch	16
Sync Pulse (HSYNC)	96
Back Porch	48
Total Horizontal Pixels	**800**

Vertical Timing (in lines)

Visible Area (Active Video)	480
Front Porch	10
Sync Pulse (VSYNC)	2
Back Porch	33
Total Vertical Lines	**525**

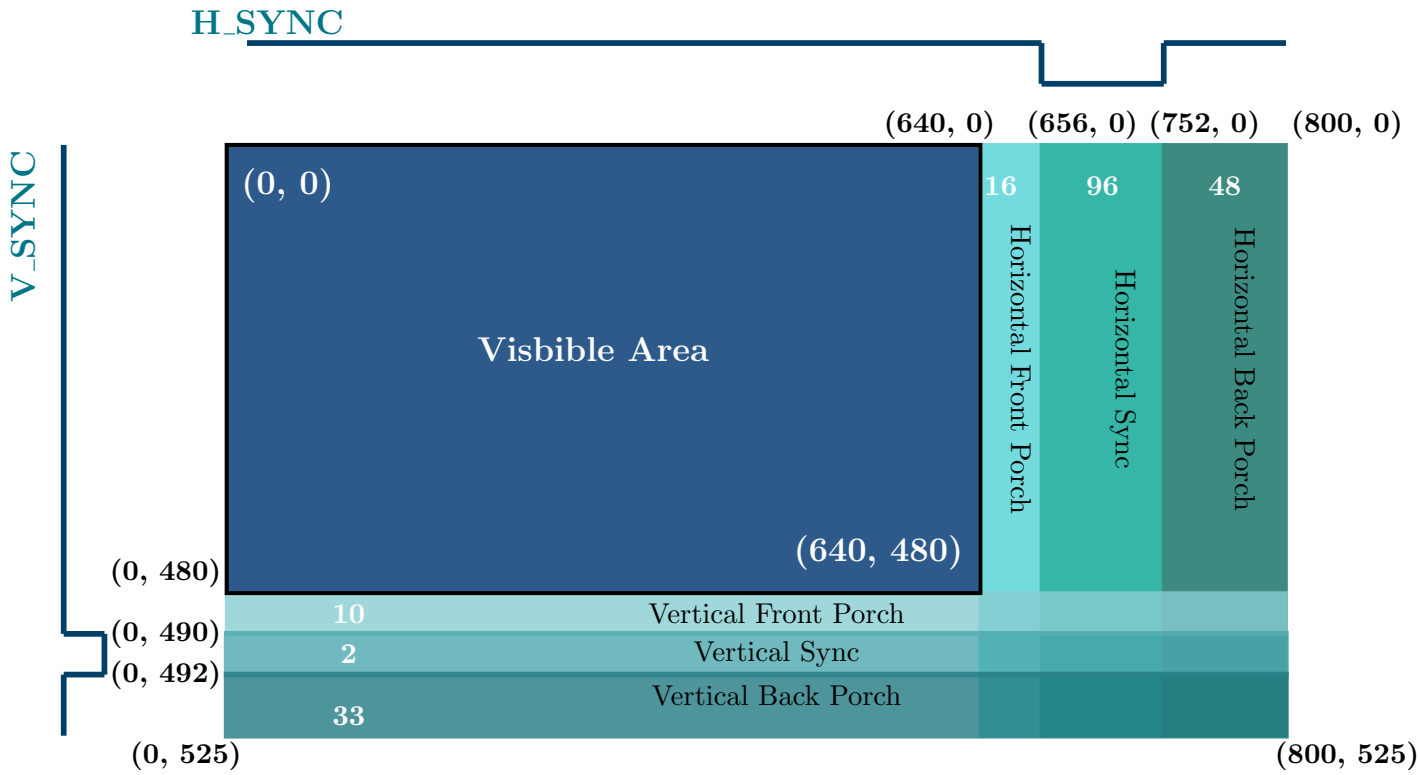

Figure 2: VGA 640×480 Timing Diagram

Signal Outputs

The VGA controller generates:

- **H_sync:** Horizontal sync pulse
- **V_sync:** Vertical sync pulse
- **V_en:** Video enable (high during visible area)
- **RGB:** Pixel color value

These are fed into the HDMI IP core to enable display on an external monitor.

2 Scrolling Background System

The background system supports infinite horizontal scrolling using tile repetition

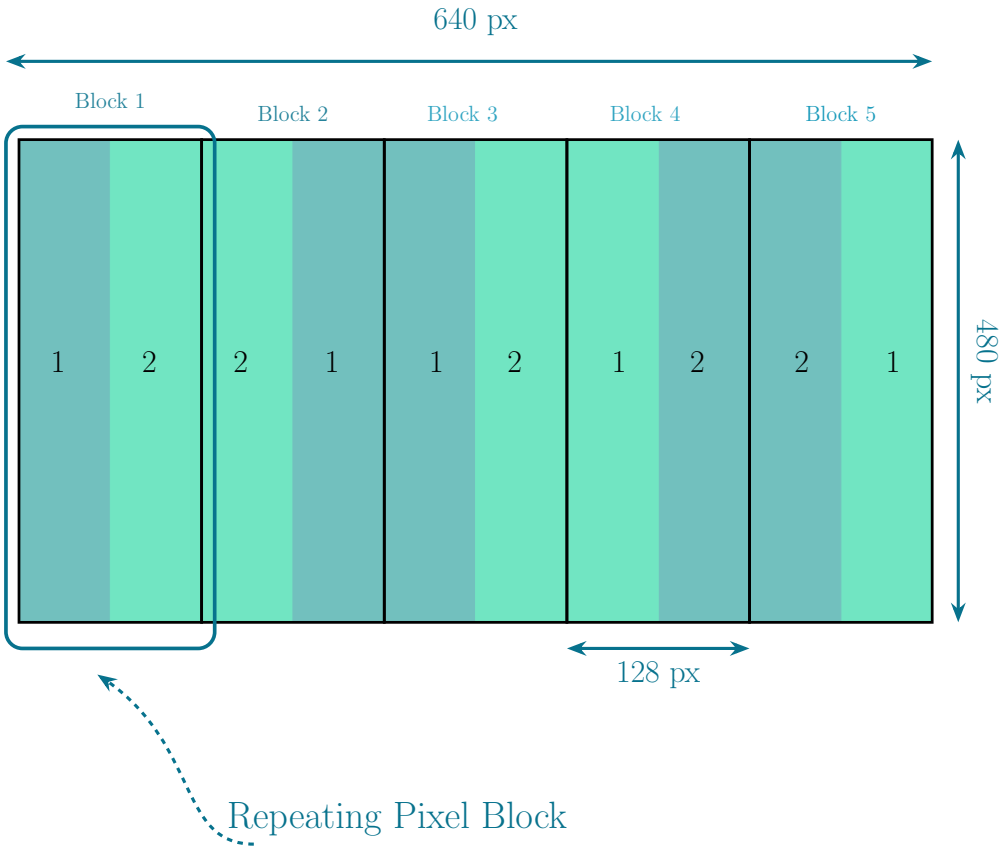

Figure 3: Infinite Backround setup

The idea involves storing a **128 pixel** wide strip in Single Port ROM (instantiated from BRAM) and mirroring and repeated placing. This allows the screen to be composed of **5** such blocks and can be seamlessly animated to move left or right. The overall ROM structure is shown below. Also, the 4 bits width is due to the fact that we use a 4 bit **Color Lookup-Table (CLUT)**, that maps 4 bit color indices to suitable **RGB565** format (16 bit), which we will be using in our project.

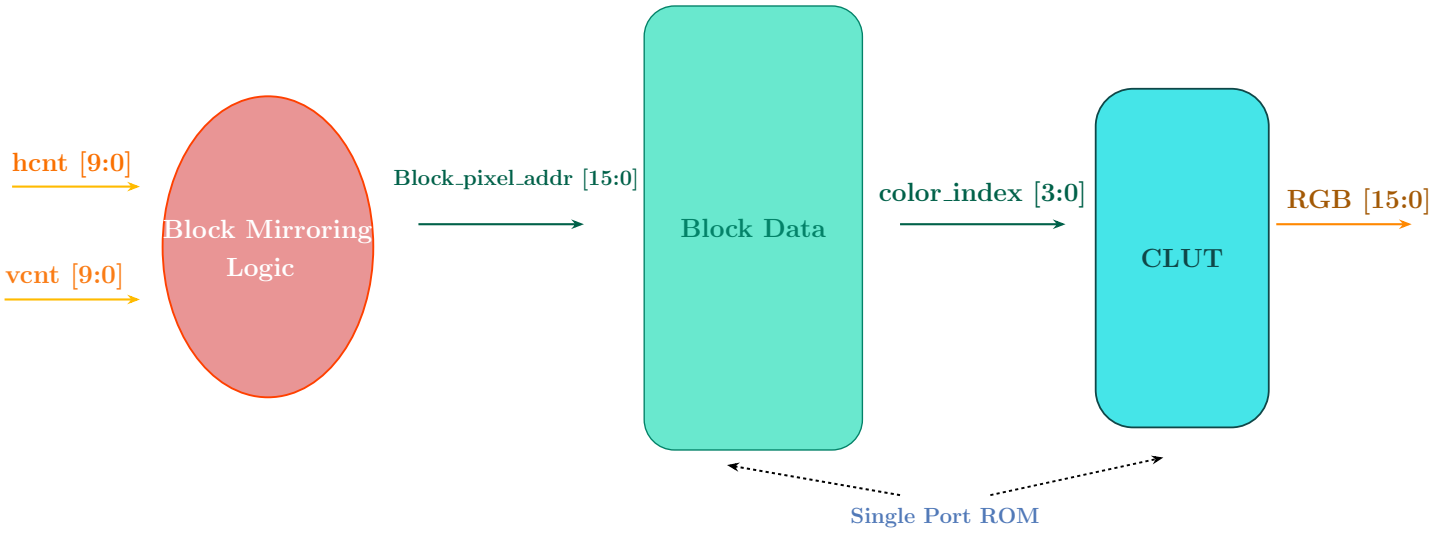

Figure 4: Overall pipeline

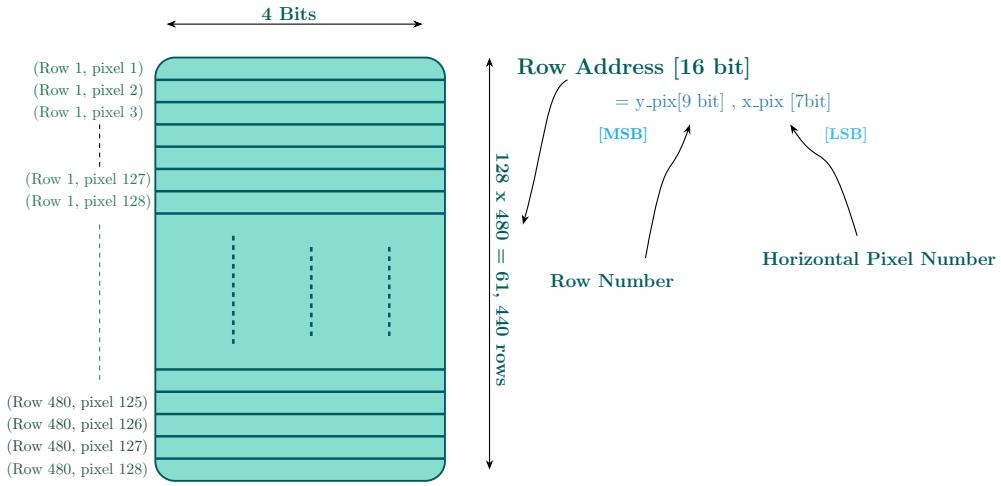

Figure 5: Block pixel data ROM structure

HDL code

```
1
2  `timescale 1ns / 1ps
3
4  module bg(
5     input clk,
6     input rst,
7     input wire [9:0] x_pix,
8     input wire [9:0] y_pix,
9     input wire btn_trigger,
10    output wire [15:0] rgb_out
11     );
12
13     reg[18:0] bias;
14     always@(posedge clk)begin
15         if(rst) bias <= 18'd0;
16         else if(y_pix == 10'd485) bias <= bias + 18'd1;
17         else bias <= bias;
18     end
19
20     wire [15:0] p_cord ;
21     wire [3:0] col_cord;
22     wire [15:0] rgb_in;
23     wire [9:0] x_pix_;
24     assign x_pix_ = x_pix + (bias>>10)-1;
25     assign p_cord = x_pix_[7] ? {y_pix[8:0],~x_pix_[6:0]} : {y_pix[8:0],x_pix_[6:0]};
26     assign rgb_out = rgb_in;
27
28 blk_mem_gen_5 rgb_value (
29   .clka(clk),    // input wire clka
30   .ena(1'b1),      // input wire ena
31   .addra(col_cord),  // input wire [3 : 0] addra
32   .douta(rgb_in)  // output wire [15 : 0] douta
33 );
34
35 blk_mem_gen_6 rgb_cord (
36   .clka(clk ),    // input wire clka
37   .ena(1'b1),      // input wire ena
38   .addra(p_cord),  // input wire [15 : 0] addra
39   .douta(col_cord)  // output wire [3 : 0] douta
40 );
41 endmodule
```

Listing 1: Scrolling background

3 Tile Management System

Each tile is a **32×32** pixel block, where each pixel is represented by a **4-bit color index**. This index is used to fetch the actual **RGB565** color from a **Color Lookup Table (CLUT)**, implemented as a **single-port ROM**.

The **Tile Management System** enables rendering of arbitrary tile-based configurations or maps as defined in the `Tile_Map`, which is a memory-mapped structure referencing up to **32 unique tiles** (5-bit indices). The system consists of the following key components:

1. **Tile Data** : stored in a **single-port ROM**, containing pixel data for all 32 tiles.
2. **Tile Map** : stored in a **single-port ROM** with a **width of 5 bits** and a **depth of 315 rows**. This allows it to represent a full screen of **20×15 tiles** (300 tiles), along with **15 extra entries** reserved for the special purpose of seamless scrolling, which will be explained later.

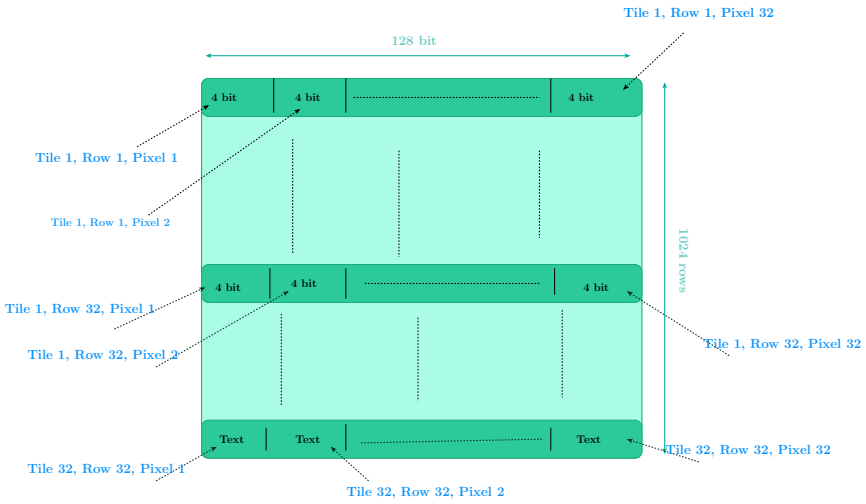

Figure 6: Tile Data (Single Port ROM)

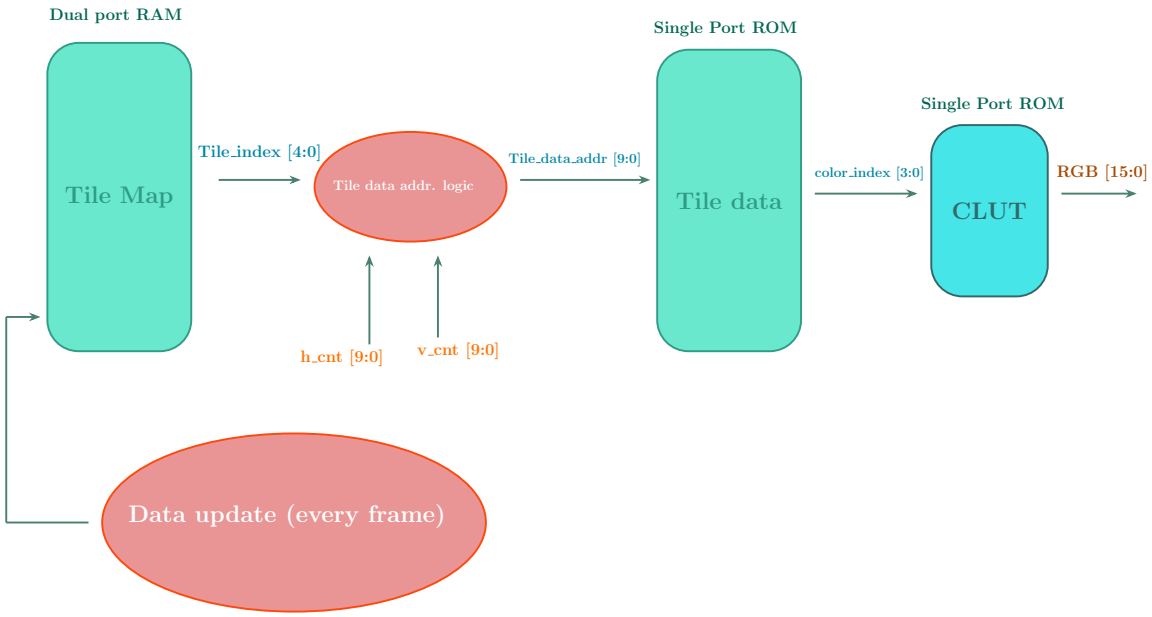

Figure 7: Overall Tile Data flow Diagram

4 Rendering Control Flow

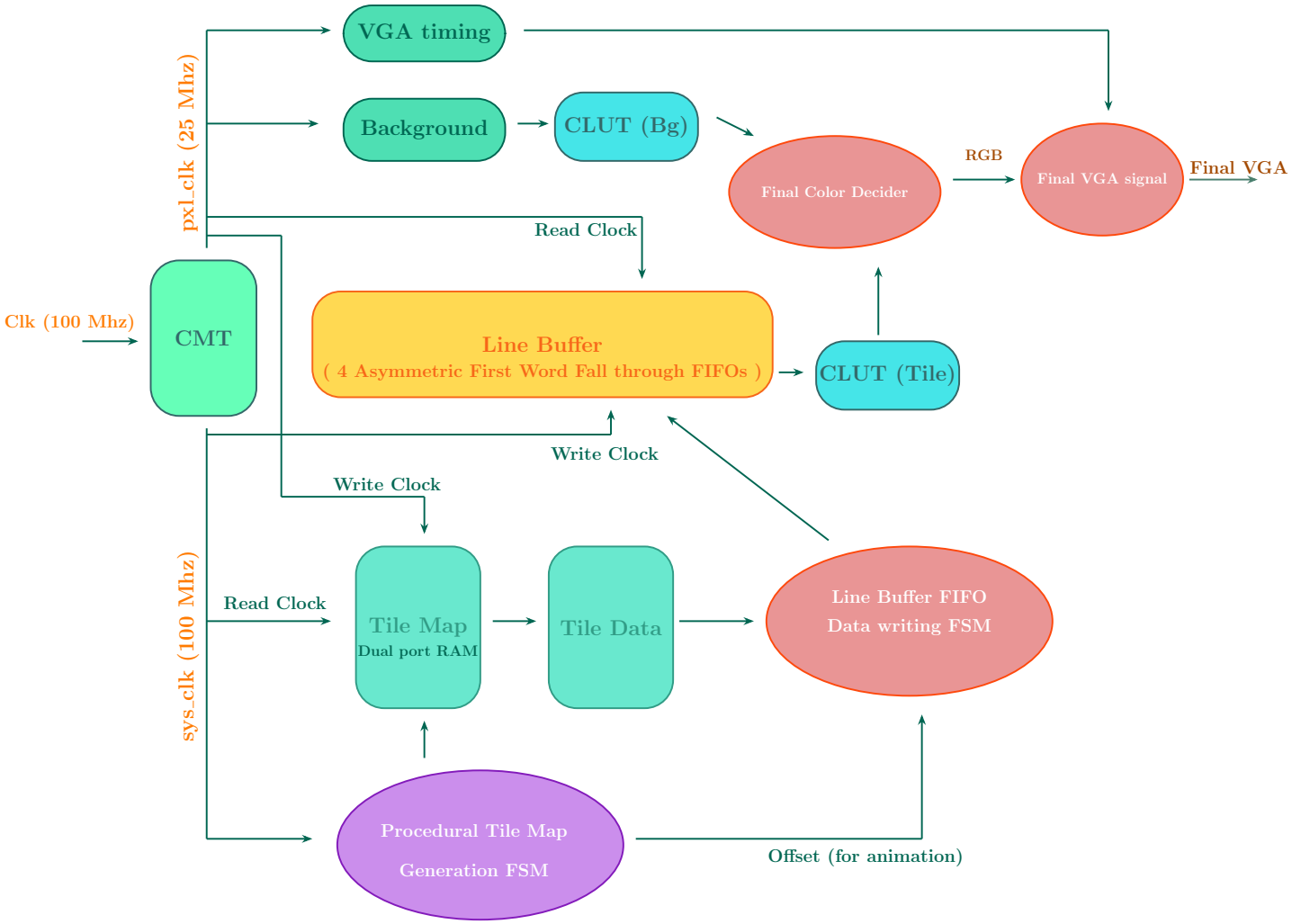

Figure 8: Rendering pipeline

Key Features of the Rendering Pipeline

- **Asymmetric FIFO System:** A set of 4 asymmetric FIFOs enables writing 128 bits (representing 32 pixels) in a single cycle, while allowing 4-bit pixel-wise reads at the pixel clock rate.
- **High-Speed Framebuffer Updates:** Leveraging a cross-clock domain design, the system writes to the framebuffer at 4× the pixel clock speed. This, combined with the 32-pixel burst writes, allows an entire 640-pixel line (2560 bits) to be flooded in just 20 system clock cycles (or 5 pixel clock cycles).
- **Massive Write Throughput:** This architecture achieves a write speed up to **128× faster** than a naive pixel-clock-based approach.
- **Efficient Memory Usage via CLUT:** By storing only 4-bit color indices and mapping them to full RGB565 values using a CLUT, the system achieves significant memory savings without compromising visual fidelity.
- **Hardware Scrolling Support:** A dedicated FSM handles writing to the line buffer with programmable offsets provided by the animation engine, enabling smooth hardware scrolling even when tile maps are only 32 pixels wide.
- **Parallelism and Timing Decoupling:** The decoupled write/read clocks and FIFO buffering allow parallel processing between tile decoding and actual pixel rendering, improving system efficiency and scalability.

5 Procedural Level Generator

Procedural Tile Generation using LFSRs

To procedurally generate game tiles, we employ multiple **Linear Feedback Shift Registers (LFSRs)** to introduce pseudo-randomness in both tile appearance and height.

- A **5-bit LFSR** is used to generate a pseudo-random height for each new tile at every frame cycle.
- The height is clamped within a range determined by the player's maximum jumping capability (typically within one tile unit).
- Based on the LFSR output, a tile type is probabilistically selected using the mapping in Table 1.
- Another LFSR controls whether a tile should be drawn at a given position, ensuring randomness while adhering to a constraint of at most **two consecutive empty tiles**.

Table 1: Tile Type Selection Probabilities

Tile Type	Probability	LFSR Output Range
Type 1	$\frac{1}{32}$	31
Type 2	$\frac{5}{32}$	26–30
Type 3	$\frac{10}{32}$	16–25
Type 4	$\frac{16}{32}$	0–15

But after choosing the tile its not final, a validation system checks if that tile is allowed as per validation rules of the game.

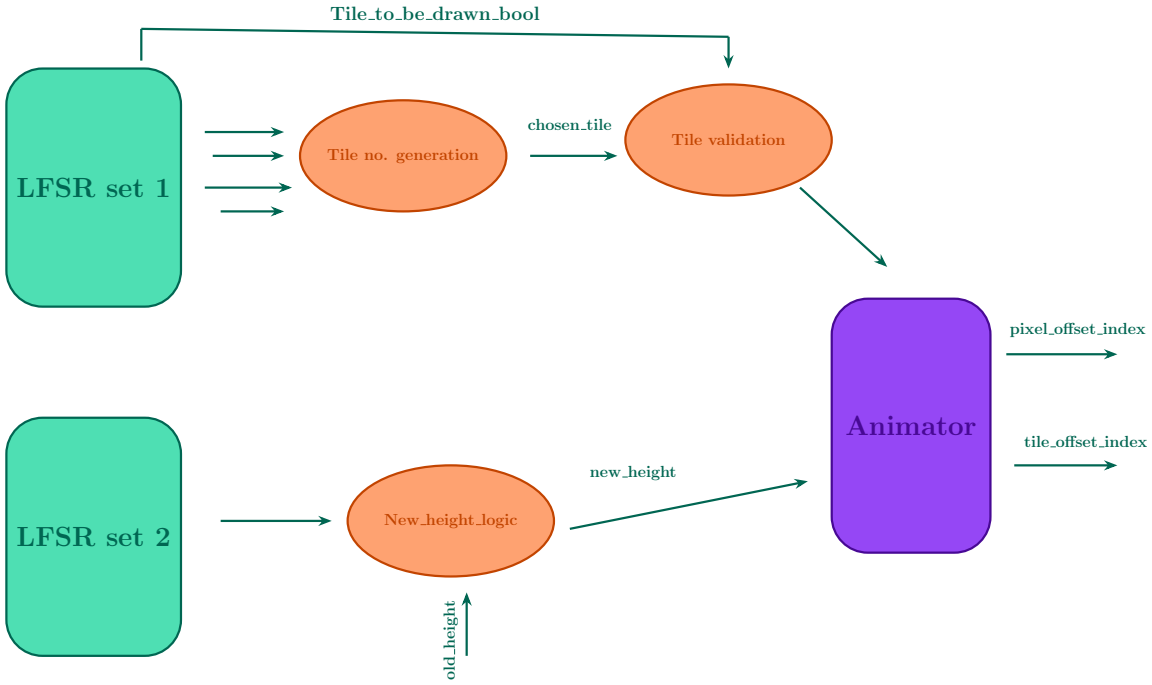

Figure 9: Overall Tile generation flow

Figure 10: Tile Level generator in action

6 Audio Output System

The `music` module is responsible for generating a melody through a passive buzzer connected to the FPGA. It operates by toggling a digital output pin at a frequency corresponding to musical notes. The system uses a 100 MHz clock to produce accurately timed square waves.

Parameter	Value
FPGA Clock Frequency	100 MHz
Output Device	Passive Buzzer
Signal Type	Square Wave
Tone Resolution	Integer Half-Periods
Counter Width	27 bits

Table 2: Audio Generator Hardware Specifications

Each musical note is encoded using a precomputed half-period value based on the 100 MHz clock. For example, to generate an E5 note (659.3 Hz), the buzzer toggles every 75,871 clock cycles, producing a square wave at the desired frequency.

Note	Frequency (Hz)	Half-Period (clock cycles)
C5	523.3	95556
D5	587.3	85127
E5	659.3	75871
F5	698.5	71633
F#5	739.9	67567
G5	783.9	63776
A5	880.0	56818
B5	987.8	50619
C6	1046.5	47778
REST	–	0

Table 3: Note to Half-Period Mapping

As an example, the following 16-note melody is implemented using this technique, with each note assigned a fixed duration:
This structure can be easily adapted to create custom melodies by changing the note sequence and duration values.

Tone Generation Logic

Two counters, `count1` and `count2`, are used:

- `count1` tracks the overall duration of a note (i.e., `stay_time`).
- `count2` toggles the buzzer output every half-period to generate the square wave.

When `count1` exceeds `stay_time`, the module transitions to the next note in the melody.

- The buzzer produces audible tones by toggling at specified note frequencies.
- Square wave generation ensures compatibility with standard passive buzzers.
- The module loops the melody indefinitely, restarting after state 15.

7 UART Input Handler

Player input is received via UART communication from a PC keyboard connected to the FPGA board. The FPGA acts as a UART receiver, interpreting incoming serial data as player commands. The UART module is clocked using a 100 MHz FPGA clock, with a baud rate of 9600 configured through the `CLKS_PER_BIT` parameter.

The UART receiver module (`uart_rx`) implements a state machine to decode incoming serial data. It samples the data line at the midpoint of each bit period to ensure accurate reception. Once all 8 data bits are received and the stop bit is validated, a valid byte signal is raised for one clock cycle.

Baud Rate Configuration

The `CLKS_PER_BIT` parameter defines the number of clock cycles per UART bit. For a 100 MHz clock and 9600 baud rate:

$$\texttt{CLKS\_PER\_BIT} = \frac{100\,\text{MHz}}{9600} \approx 10417$$

State Machine

The design uses five states:

- `s_IDLE` – Waits for the start bit (logic 0)
- `s_RX_START_BIT` – Confirms valid start bit at midpoint
- `s_RX_DATA_BITS` – Samples 8 data bits, LSB first
- `s_RX_STOP_BIT` – Validates stop bit (logic 1)
- `s_CLEANUP` – Raises valid data signal and resets state

Behavioral Summary

- Double-registering is used for metastability protection.
- The received byte is made available on `o_Rx_Byte`.
- A one-clock pulse on `o_Rx_DV` indicates valid data.

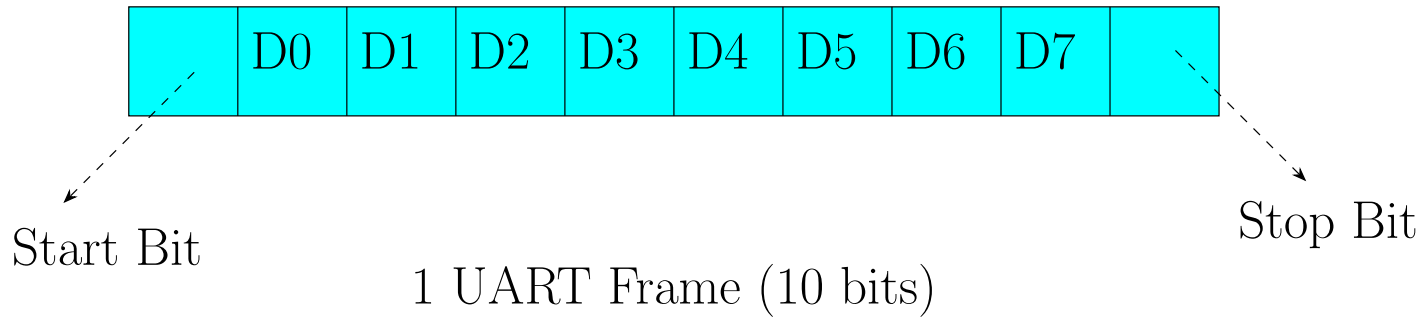

Figure 11: UART Frame: Start bit, 8 data bits, Stop bit

Source: This UART receiver Verilog code was adapted from the tutorial provided on nandland.com.

8 BCD-Based Timer System

The `simple_counter` module implements a Binary-Coded Decimal (BCD) timer that counts time in seconds. It is compatible with a 100 MHz FPGA clock and includes pause and reset functionality. The time is represented in a 16-bit output `count5`, structured as four 4-bit BCD digits, which can be mapped directly to a 4-digit 7-segment display.

Functional Description

This timer counts from 0000 to 9999 in BCD format, incrementing every 1 second based on a clock divider. Key features include:

- **Pause Control:** The timer halts incrementing when `pause` is asserted.
- **Reset Control:** Resets both the internal clock divider and the BCD output when `rst` is high.
- **Clock Divider:** A 28-bit counter is used to divide the 100 MHz clock down to 1 Hz.
- **Decimal Overflow Handling:** Each BCD digit rolls over from 9 to 0 and carries to the next higher digit.

Output Format

The 16-bit output `count5` is structured as follows:

`[15:12] : [11:8] : [7:4] : [3:0] = D3 : D2 : D1 : D0`

Each nibble represents a decimal digit (0–9), allowing human-readable display on four 7-segment digits.

HDL Code

```
module simple_counter(
    input clk,
    input rst,
    input pause,
    output reg [15:0] count5
);

reg [27:0] count;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        count <= 28'b0;
        count5 <= 16'b0;
    end else begin
        // Increment only when not paused and not at 9999
        if (~pause && ~(count5[3:0]==4'd9 && count5[7:4]==4'd9
            && count5[11:8]==4'd9 && count5[15:12]==4'd9)) begin

            if (count < 28'd99999999)
                count <= count + 1;
            else begin
                // BCD digit cascading logic
                if (count5[3:0] < 4'd9)
                    count5[3:0] <= count5[3:0] + 1;
                else if (count5[7:4] < 4'd9) begin
                    count5[7:4] <= count5[7:4] + 1;
                    count5[3:0] <= 4'd0;
                end else if (count5[11:8] < 4'd9) begin
                    count5[11:8] <= count5[11:8] + 1;
                    count5[7:4] <= 4'd0;
                    count5[3:0] <= 4'd0;
                end else if (count5[15:12] < 4'd9) begin
                    count5[15:12] <= count5[15:12] + 1;
                    count5[11:8] <= 4'd0;
                    count5[7:4] <= 4'd0;
                    count5[3:0] <= 4'd0;
                end
                count <= 28'b0;
            end
        end
    end
end

endmodule
```

Listing 2: BCD Timer Verilog Code

9 UI System (for score display)

The scoring mechanism in this platformer game is implemented using a **16-bit BCD counter** and a custom digit-rendering pipeline. The score is incremented over time unless the system is paused or reset, and it is displayed on the screen using 32×32 monochrome bitmaps for each digit.

Components Involved

- **Timer Counter (`simple_counter`):** Increments a 16-bit BCD score every `100 million` clock cycles when not paused.
- **Digit Bitmap Mapper (`number_list`):** Converts each of the 4 BCD digits into corresponding 32-bit wide scanlines fetched from preloaded BRAMs.
- **Monochrome Bitmap BRAMs:** Each digit (0–9) has a dedicated BRAM initialized with 32×32 bitmaps using `.coe` files.
- **Display Renderer (`rgb_val`):** Uses the bitmaps from `number_list` to light up appropriate pixels for each digit depending on current pixel coordinates.

Counter Logic

- The score is stored in a 16-bit register `counter_nm` encoded in BCD format.
- The `simple_counter` module keeps a 28-bit internal timer that triggers score increments.
- The increment logic correctly handles BCD carry: each digit wraps from 9 to 0 and carries to the next higher digit.

Digit Rendering Pipeline

1. The 16-bit BCD score is broken into 4 digits using slicing:
   - `counter_data[15:12]` → Most significant digit
   - `counter_data[3:0]` → Least significant digit
2. For every digit, the corresponding BRAM (out of 10) is selected using a `case` statement.
3. Each BRAM outputs a 32-bit scanline (`digit_cor`) corresponding to `y_cor` (vertical pixel line).
4. During VGA rendering, the appropriate scanline and pixel bit are used to determine whether a pixel is active (white) or transparent (background).

Score Display Area

- The score appears in the top center of the screen.
- Digit positions are fixed (e.g., starting from `x=384` to `x=512`, one digit per 32 pixels).
- Each digit appears when `y_pix` is in range `[32, 64)`.
- Horizontal bit indices (for each digit) are calculated using:
  - `digit_index = ~(x_pix - offset)` where `offset` = 384, 416, 448, 480 respectively.

Figure 12: Visualization of the score in the game

COE File Format (Used for BRAM Initialization)

The bitmap for each digit is stored as a `.coe` file. Below is an example snippet for the digit '0':

```
memory_initialization_radix=2;
memory_initialization_vector=
00000000000000000000000000000000,
00000011111111111111111111110000,
00000011111111111111111111110000,
00000011111111111111111111110000,
00000011111111111111111111110000,
00000111111111111111111111100000,
00000111111111111111111111000000,
00000000000000000000111111000000,
00000000000000000001111110000000,
00000000000000000011111100000000,
00000000000000000111111000000000,
00000000000000001111111000000000,
00000000000000011111110000000000,
00000000000000111111110000000000,
00000000000001111111100000000000,
00000000000001111111100000000000,
00000000000011111111100000000000,
00000000000011111111100000000000,
00000000000111111111000000000000,
00000000000111111111000000000000,
00000000001111111110000000000000,
00000000001111111110000000000000,
00000000011111111110000000000000,
00000000011111111110000000000000,
00000000111111111100000000000000,
00000000111111111100000000000000,
00000000000000000000000000000000;
```

Listing 3: Monochrome Bitmap COE for Digit 0

10 Sprite Animation System (Not fully implemented)

11 Physics Engine(Not fully implemented)

12 Tile Designer JavaScript Applet

This allows one to generate the required .coe files for BRAM initialisation based on the drwan 32x32 tile, making game development workflow much faster.

13 Conclusion

This project demonstrates the design of a 2D Tile based Game engine from full scratch and custom architecture in Verilog HDL, complete with most standard features.

Key Challenges Faced

1. **Ideation:** Honestly, the hardest part was just figuring out what to build. With such an open-ended project, everything was up in the air—from what the background color should be to what the gameplay should even look like. Getting from "cool idea" to an actual design architecture was no small task.

2. **Setup/Hold Violations:** We initially wrote some really bad code that unknowingly violated timing constraints. It took us way too long to realize that this was the root cause of weird bugs. Debugging that mess ate up a good chunk of our time.I had to rewrite the entire tile rendering system because of this.

3. **Cross-Clock Domain Issues:** We had the idea of using a faster clock for writing data from the very beginning, but implementing it correctly was another story. Specifically, sending signals cleanly from the pixel clock domain to the system clock domain (like triggering data writes and animations) caused a lot of pain.

4. **Animation FSM :** The animation finite state machine was one of the most complex parts. We wrote a "one-shot" version of it thinking we were clever—but it just ended up giving us a solid blue screen. Took straight 4-5 hrs to debug and that, turns out there were multiple faults from integer overflow to subtle timing issues wih the FIFO etc.

Major Learnings

1. Gained practical experience with **Vivado tools**, including the Integrated Logic Analyzer (ILA), behavioral simulation, Design Rule Checks (DRC), Timing and Methodology Summaries. These were especially useful in debugging **clock domain crossing (CDC)** issues.

2. Developed a deeper understanding of how to approach problems with a **logic hardware mindset**.

3. Realized the importance of a **systematic and structured design methodology**.

14 Bibliography

Use of AI

AI (specifically large language models) played a significant role in the development of the following aspects of this project:

1. **Code debugging or generation**: I would have been grateful if it were that useful, wasted way too many hours over dumbly written code.
2. **LaTeX Documentation** : AI assisted with formatting and refining the content for better clarity and presentation.
3. **JavaScript Applet for `.coe` Generation** : Helped design and structure the code to create tile memory initialization files.
4. **Idea Formulation** – While AI was not particularly helpful for writing or debugging Verilog or low-level logic, it contributed to refining and articulating a few design ideas more clearly.

Resources of Impeccable Value

Several external resources proved invaluable throughout this project:

- fpga4fun.com
- projectf.io
- nandland.com
- TikZMaker – Used for drawing elegant LaTeX diagrams.
- **Vivado Tools** : Especially the ILA, Linter, and Methodology Summary features were extremely useful during implementation and debugging. Also the timing summary helped us realise the need for pipelining

Github : While no significant part or architecture of the code is copied (or even influenced by any GitHub repository), some sections as mentioned earlier too (like UART) have received significant help from websites as mentioned.